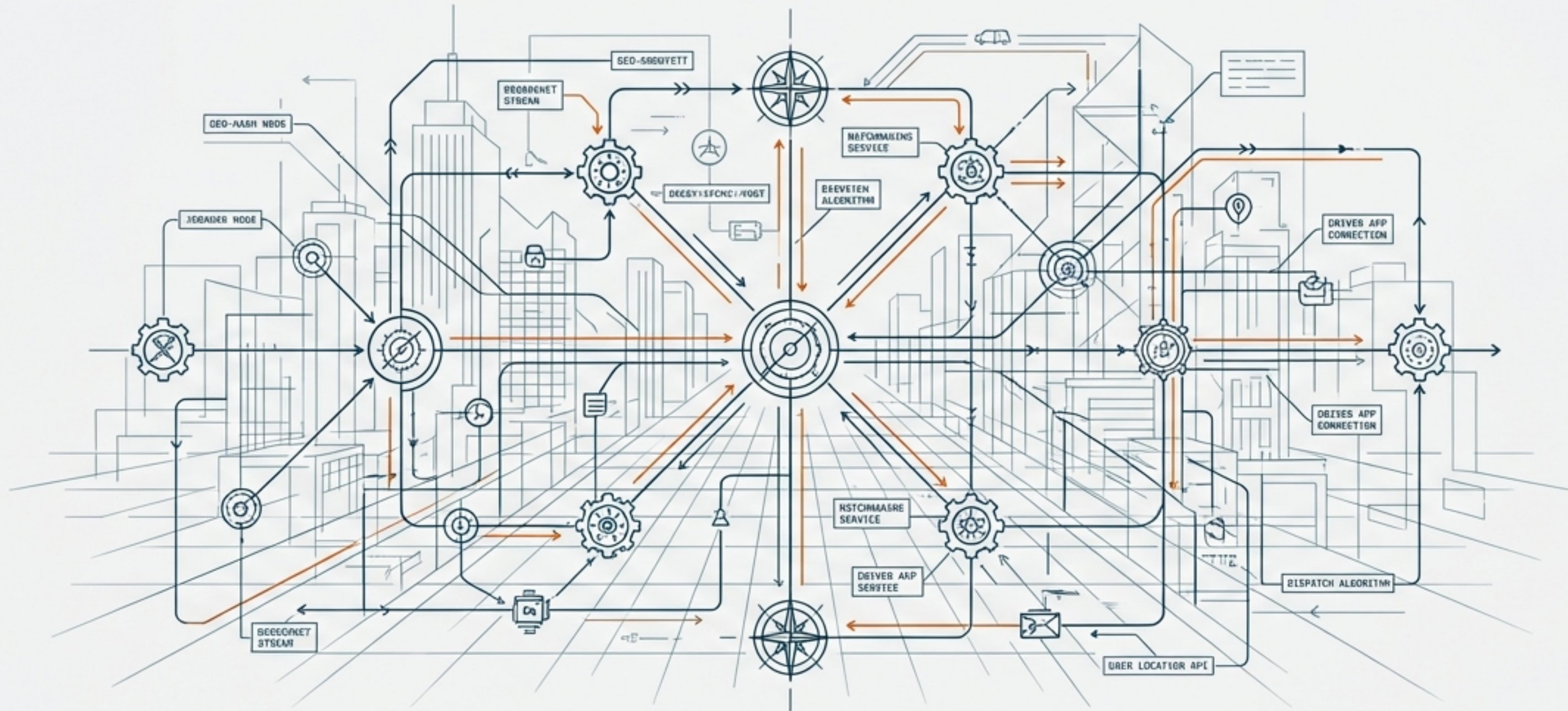


ARCHITECTING REAL-TIME RIDE-HAILING

A COMPREHENSIVE STUDY GUIDE



FROM GEO-HASHING TO WEBSOCKET ORCHESTRATION



Defining the Scope: Functional & Non-Functional Requirements

FUNCTIONAL (The What)



Driver Discovery: Identify supply in milliseconds.



ETA Calculation: Accurate arrival prediction.



Location Tracking: Real-time bi-directional updates.



Dynamic Pricing: Supply/Demand cost adjustment.

NON-FUNCTIONAL (The How)



Availability: Zero downtime (0% tolerance).



Scalability: Linear growth support (20M+ rides/day).



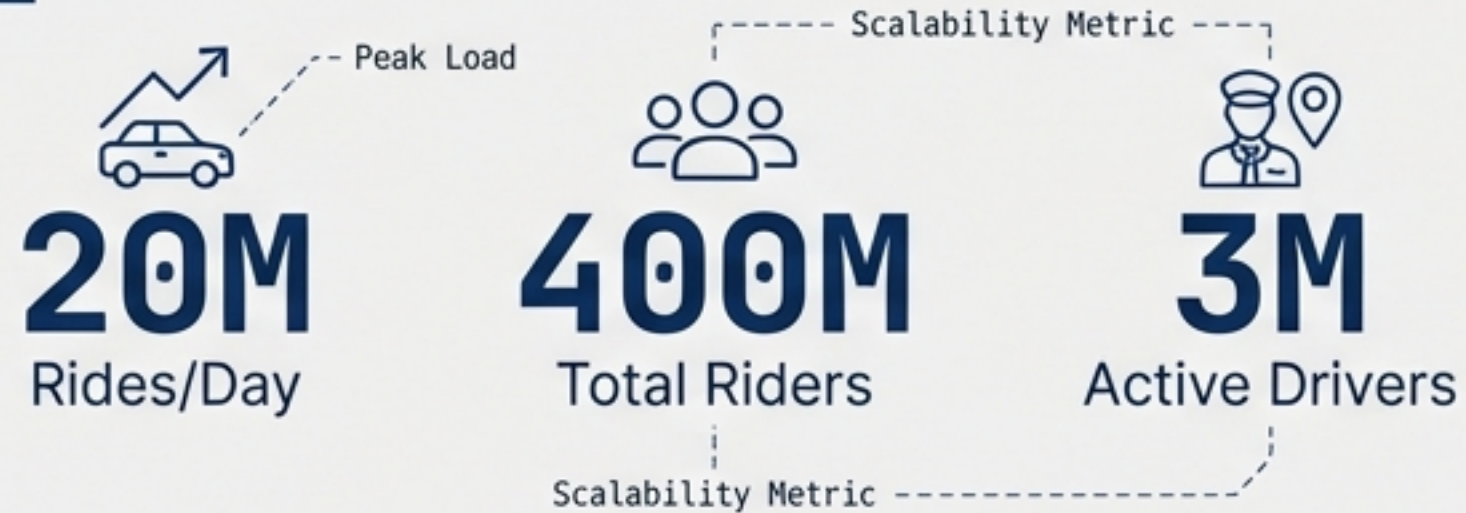
Consistency: View synchronization (Rider/Driver State).



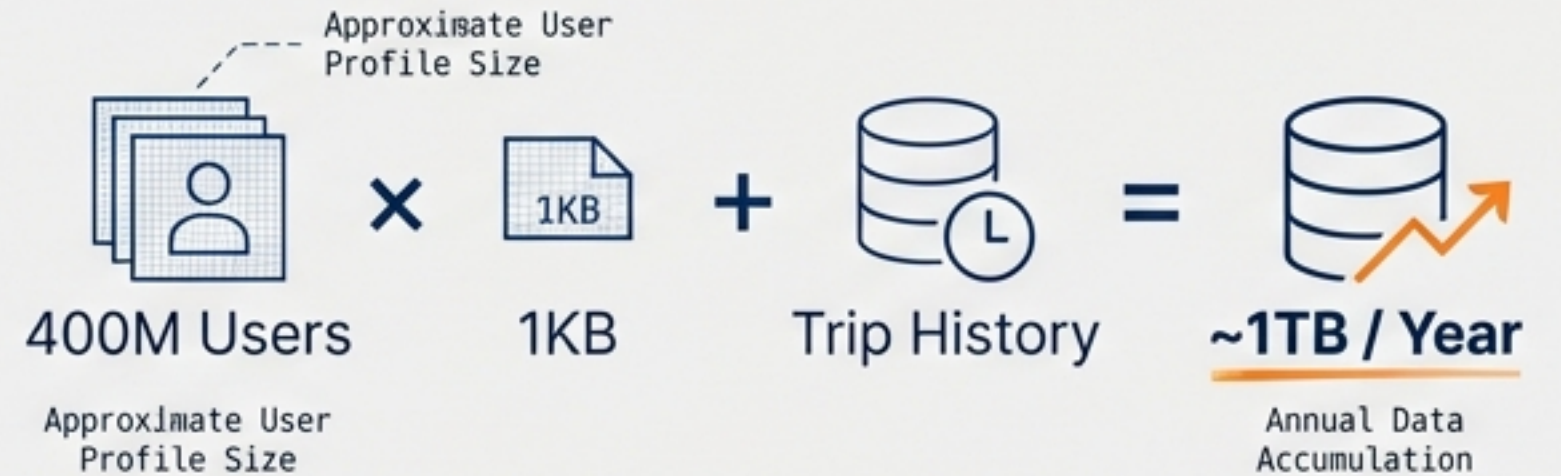
Reliability: Error-free execution.

Capacity Planning & Resource Estimation

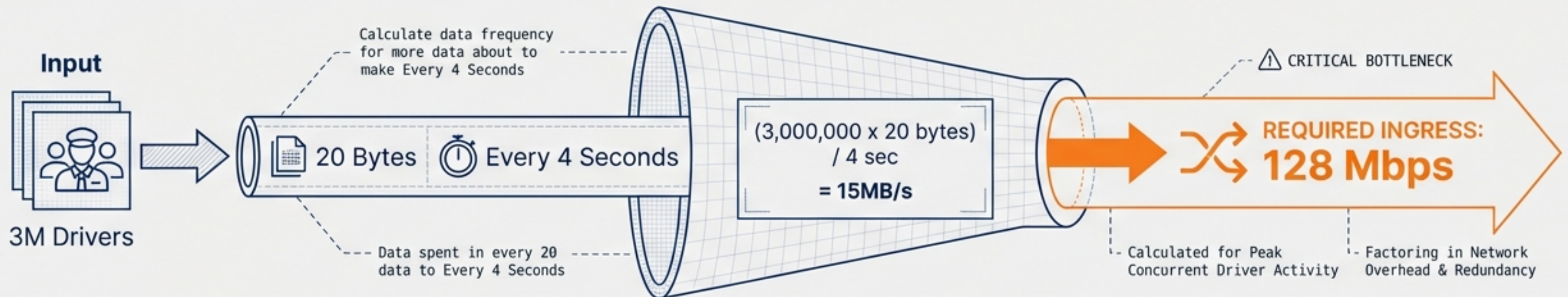
TRAFFIC VOLUME



STORAGE FOOTPRINT



BANDWIDTH (The Bottleneck)



The Database Dilemma: Why Standard SQL Fails at Scale

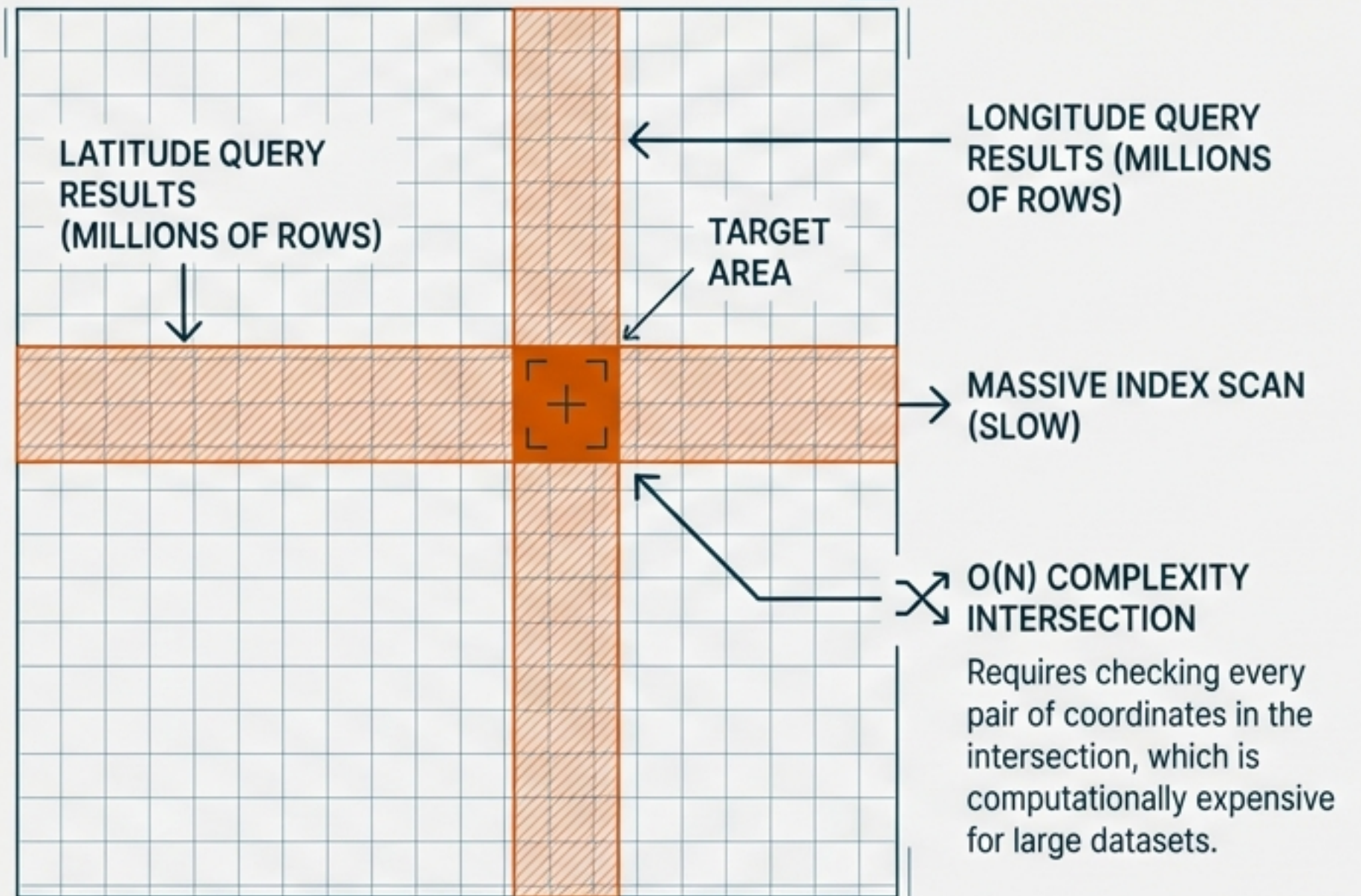


STANDARD SQL 2D RANGE QUERY

```
SELECT * FROM Drivers
WHERE lat BETWEEN (r_lat - 5) AND (r_lat + 5)
AND long BETWEEN (r_long - 5) AND (r_long + 5)
```

Directly querying latitude and longitude columns on massive tables.

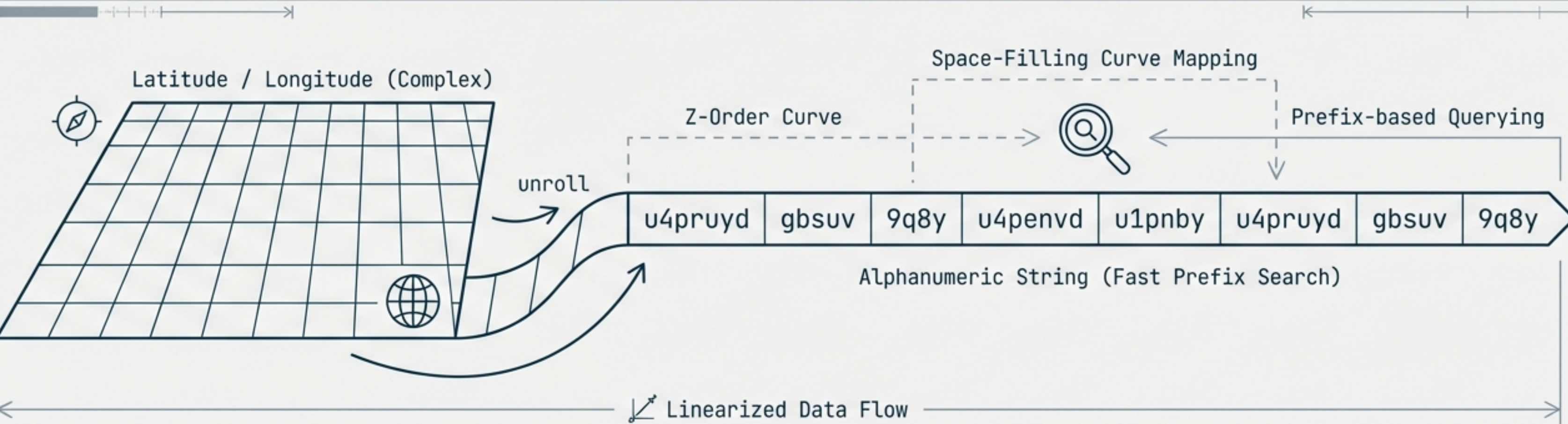
THE INTERSECTION PROBLEM



2D Range Queries on 2M+ records are too slow for real-time blocking calls.

Introduction to Geospatial Indexing

Linearizing Multi-Dimensional Data





HASH-BASED (Grid)

Even Grid

Cartesian Tiers

GeoHash (Selected Standard)



TREE-BASED (Hierarchical)

QuadTree

Google S2

RTree



The Mechanics of GeoHash: Recursive Quadrants

Geospatial Indexing Techniques - Technical Specification - REV 1.0



Recursive Zoom: Each character added to the string refines the grid location, exponentially increasing precision.

GeoHash Precision & Bit Depth

String Length	Cell Dimensions	
1 Character	5,000km x 5,000km Continent	
4 Characters	39km x 19km City	
6 Characters	1.2km x 0.6km Neighborhood	
8 Characters	38m x 19m Building - Target Precision	



Stop Condition:
Subdivide until
Grid Cell Size $\approx$
Search Radius (5km).

Base32 Encoding: The Human Layer

Raw Binary —————> Encoding Map —————> Base32 String

10111001101011001110101101110010...

Grouping

[10111][00110][10110][01110][10110][11100][10....]

5 Bits (2^5 = 32)

00000	→	0
00001	→	1
...		
11110	→	y
11111	→	z

u4pruyd

Optimized for URL safety, database storage, and human readability.

Implementation: Storing GeoHash in Redis

Operational Profile:

- Swiggy/Yelp: READ-Heavy (Static locations).
- Uber/Ride-Hailing: WRITE-Heavy (Dynamic locations, updates every 4s).

Redis (In-Memory Store)

`GEOADD driver:location 123 56.789 -56.789`

Value: Sorted Set

Score: 12870376941588750 (u4pruyd...

Score: 12870376941588751 (u4pruyd...

driver:location 123 →

Score: 12870376941588752 (u4pruyd...

Score: 12870376941588753 (u4pruyd...

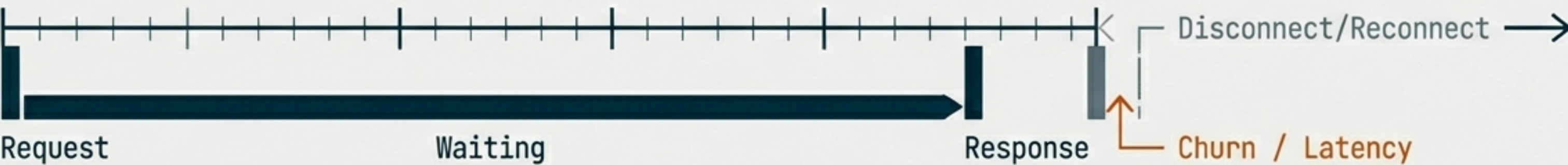
High Throughput, Low Latency.

The Communication Gap: Why HTTP & Polling Fail

HTTP Short Polling → **Inefficient**



Long Polling → **Fragile**



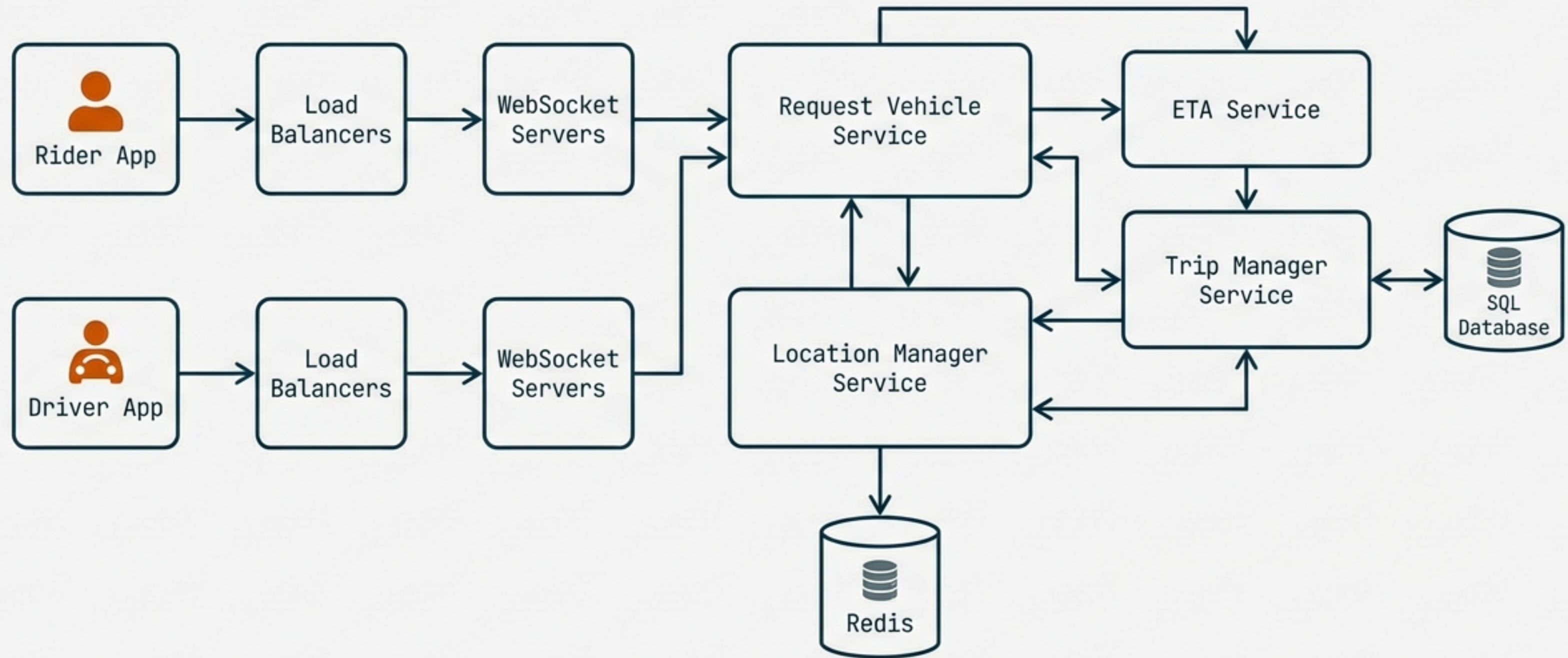
Requirement: Bi-directional, sub-second latency for moving targets.

The Solution: WebSockets (Full-Duplex Communication)

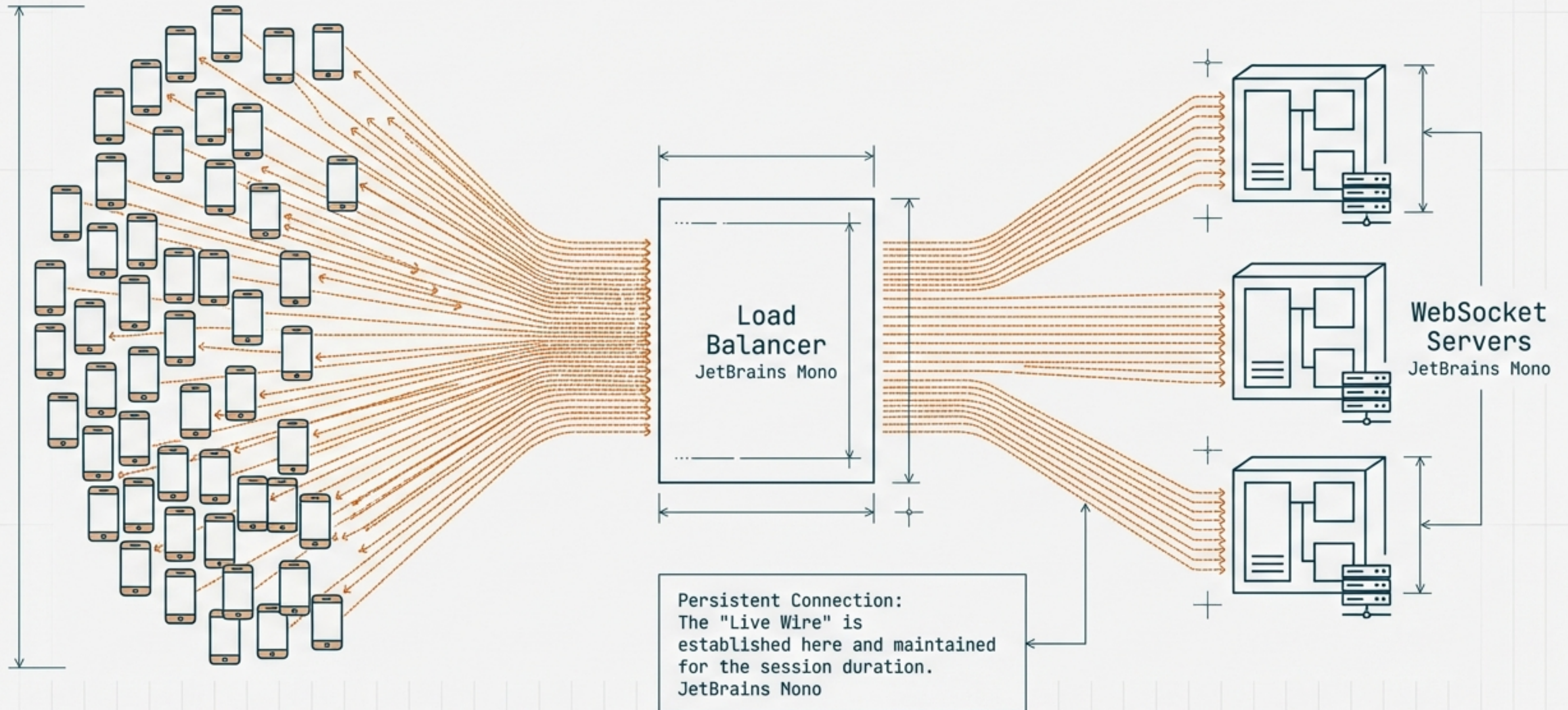


Protocol	Direction	Suitability
HTTP	Half-Duplex	Transactions
SSE (Server Sent Events)	Unidirectional (Server->Client)	Notifications
WebSocket	Full-Duplex	Real-Time Tracking (Winner)

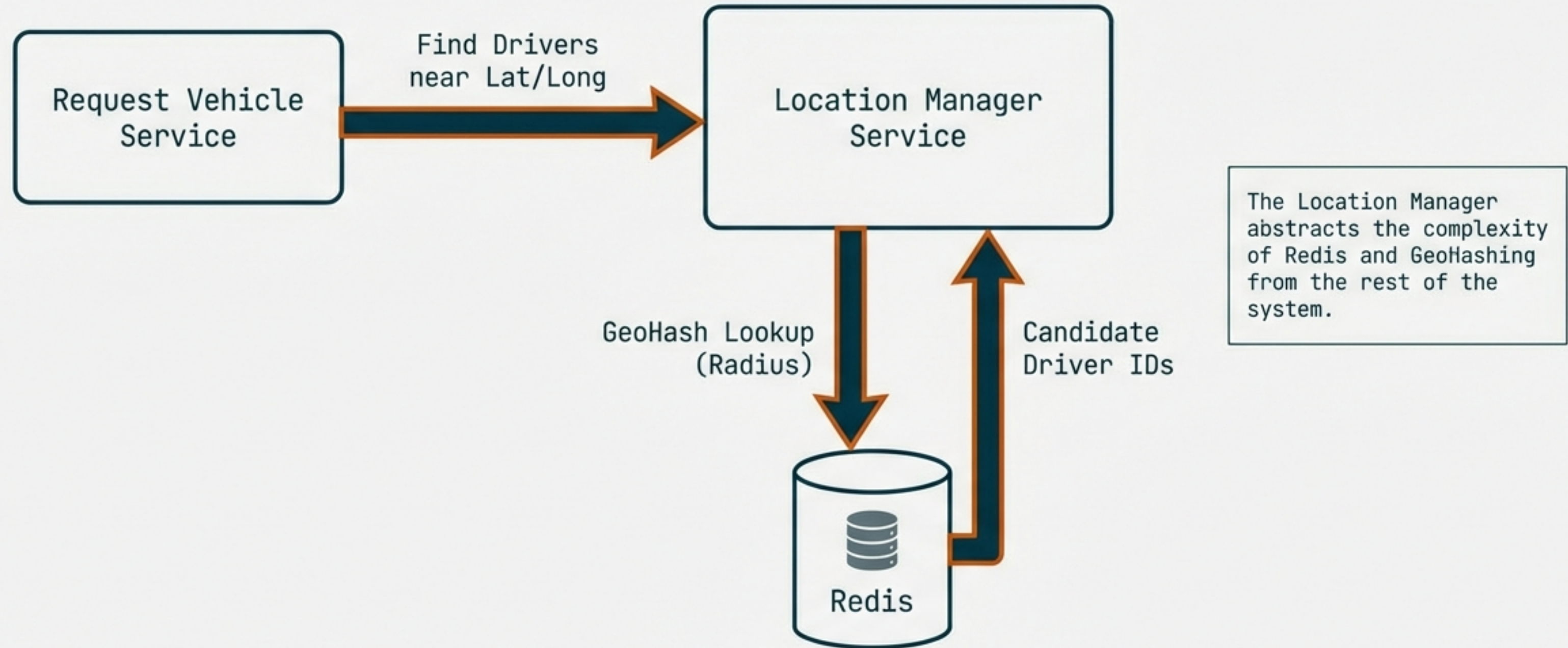
High-Level Architecture Overview



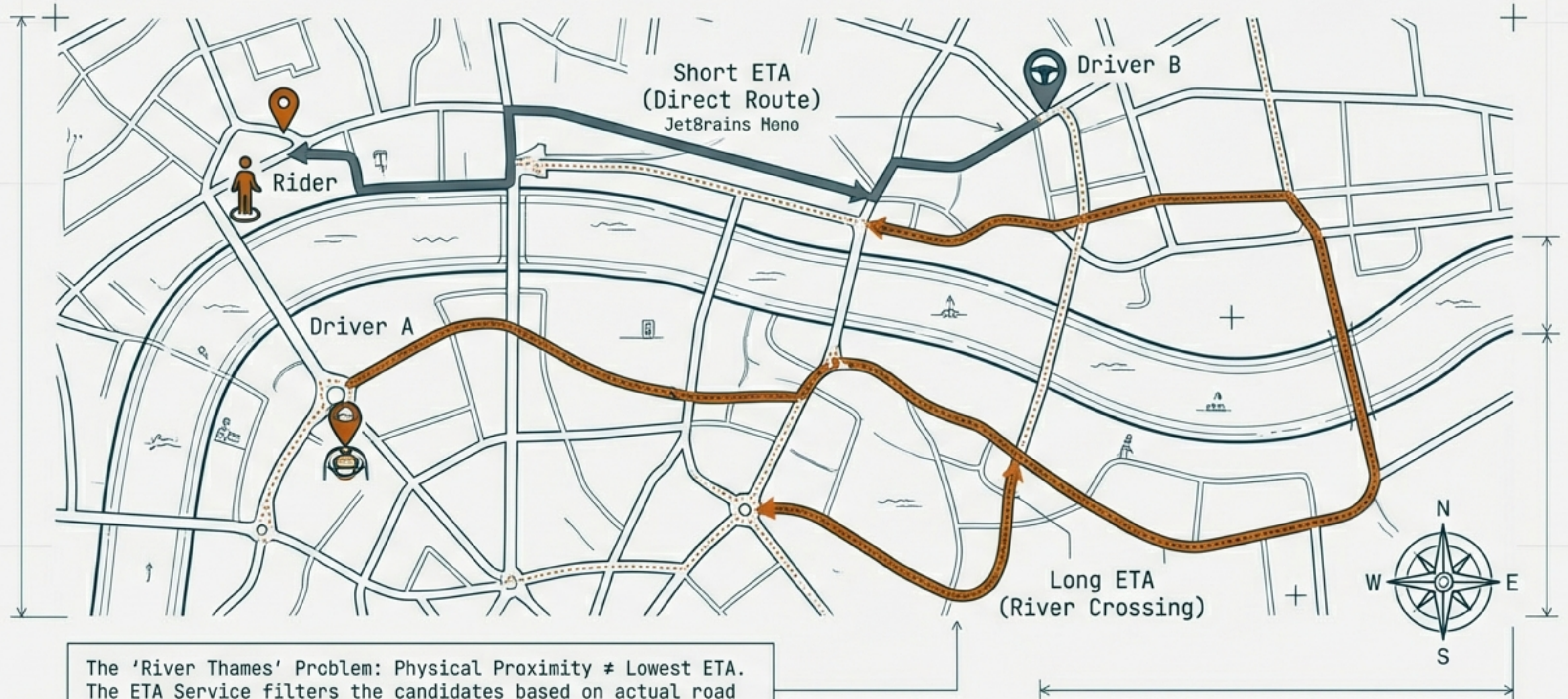
Step 1: The Entry Point (Load Balancers & Connections)



Step 2: The 'Brain' (Request Service & Location Manager)

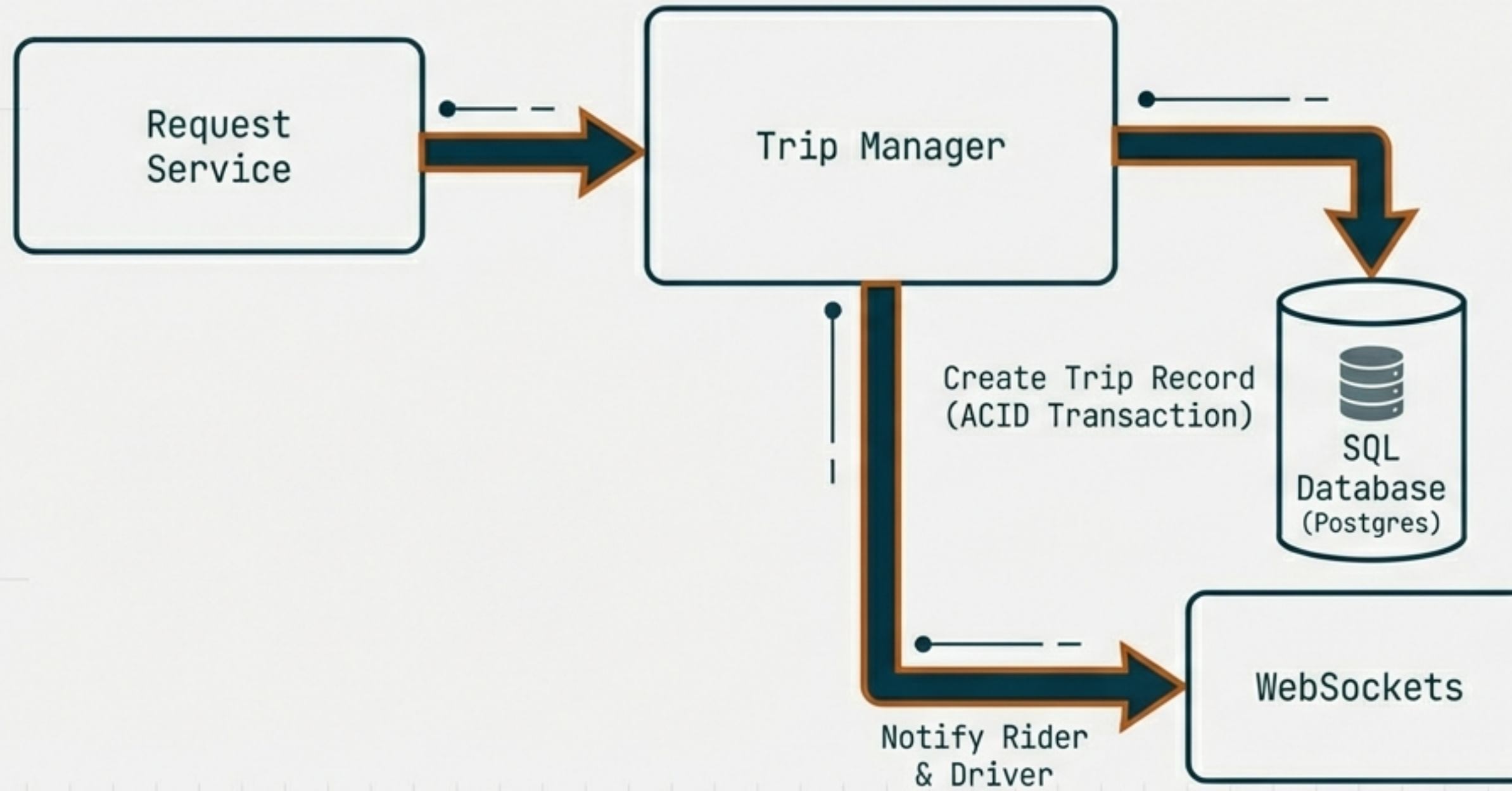


Step 3: The Decider (ETA Service & Optimization)



The 'River Thames' Problem: Physical Proximity $\neq$ Lowest ETA.
The ETA Service filters the candidates based on actual road network traversal time.

Step 4: The Record Keeper (Trip Manager & Databases)

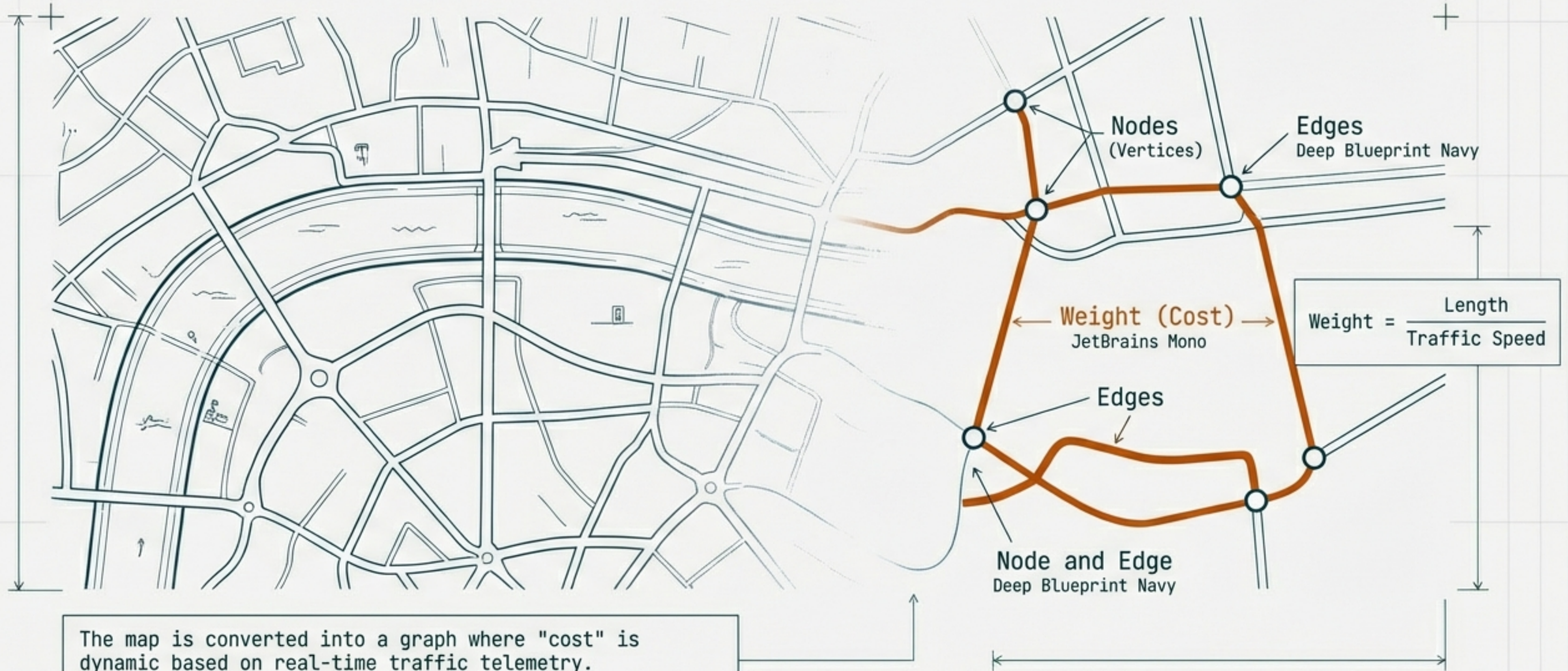


Sidebar Comparison:

Redis (Location Manager):
Volatile, Fast, Ephemeral.

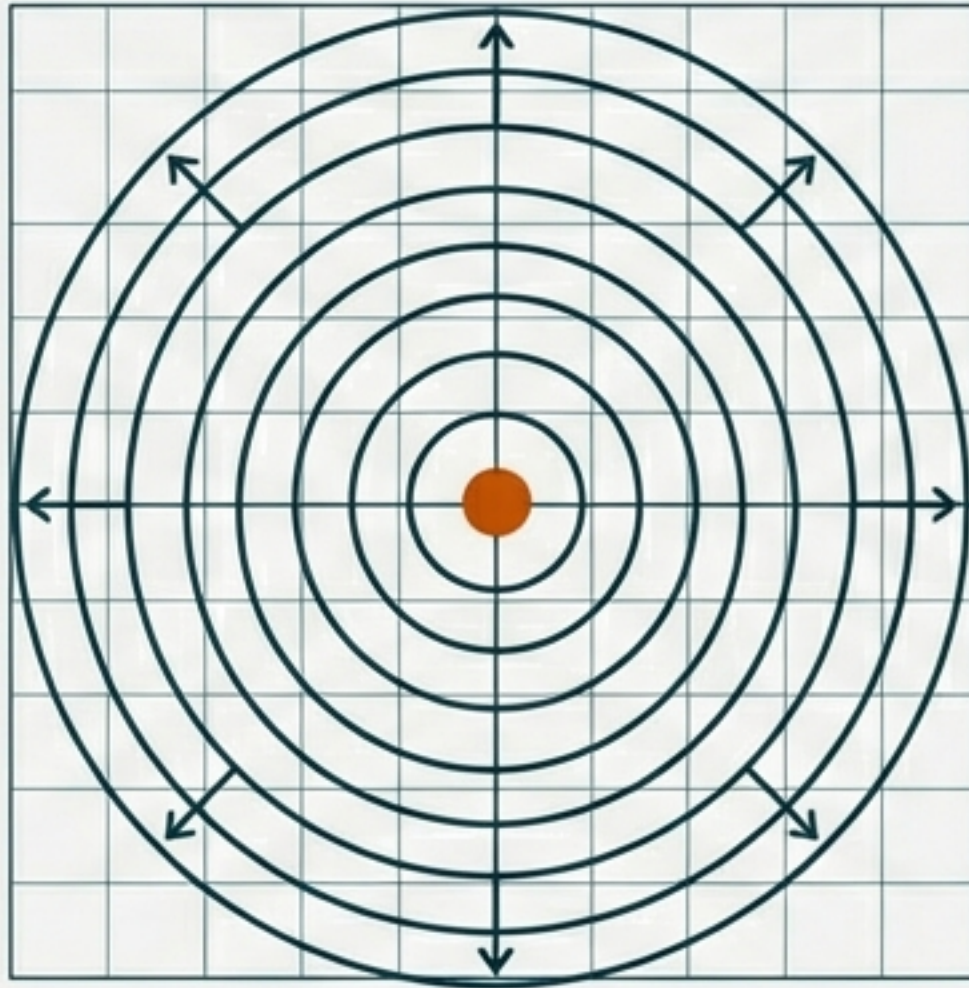
SQL (Trip Manager):
Durable, Transactional,
Billing-Critical.
JetBrains Mono

Modeling the Real World: Graph Theory



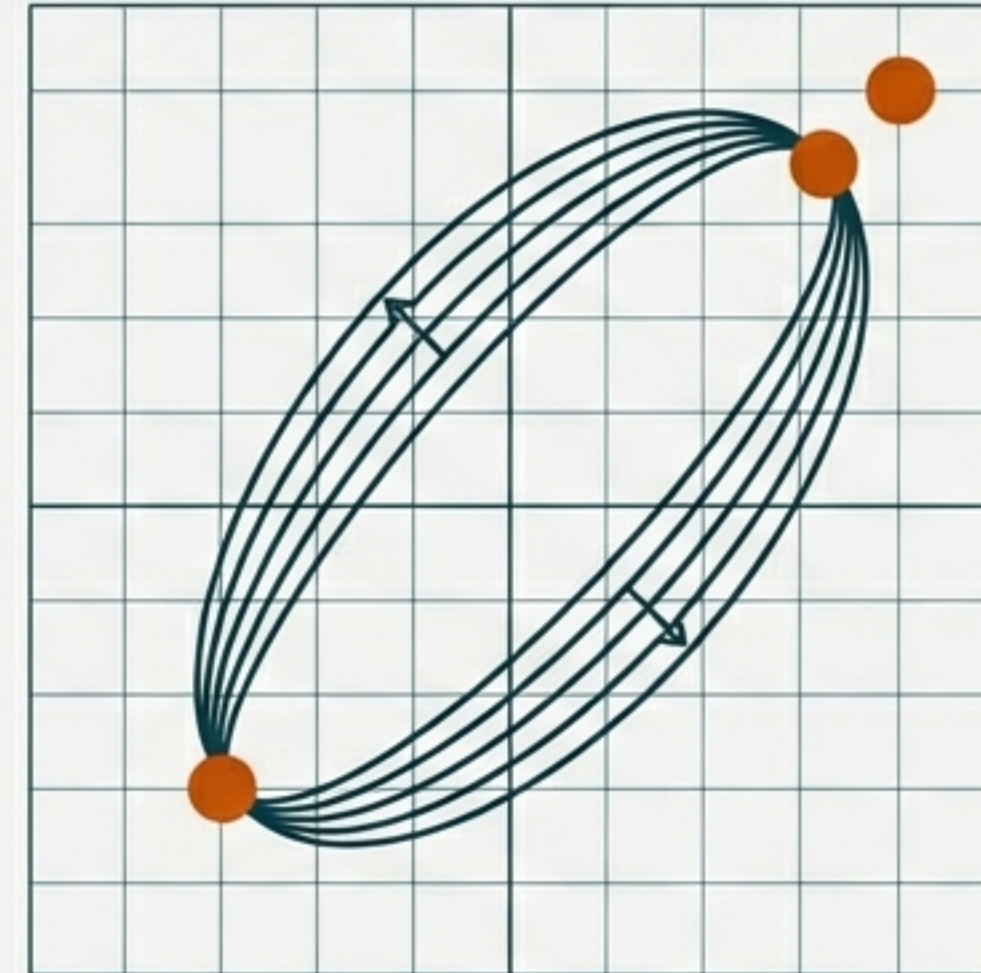
Pathfinding Algorithms: Dijkstra vs. A*

Dijkstra Algorithm



Guaranteed Shortest, Computationally Expensive.
JetBrains Mono - Deep Blueprint Navy

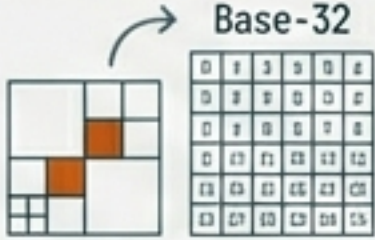
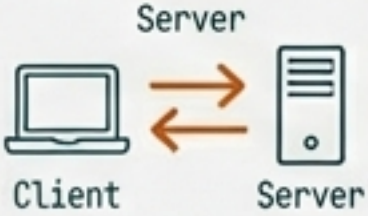
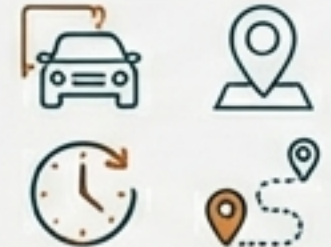
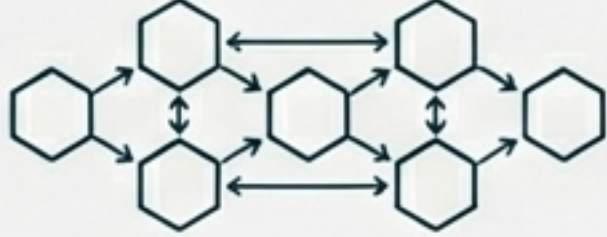
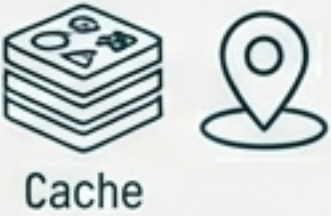
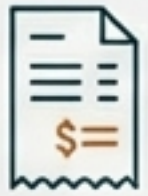
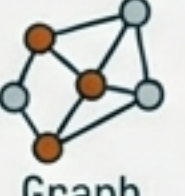
A* Algorithm



Heuristic-Driven, Directed, Faster.
JetBrains Mono - Deep Blueprint Navy

Production systems use A* with dynamic weights to minimize search space.

System Summary: The “Cheat Sheet”

INDEXING	GeoHash (Base32) JetBrains Mono		Logic: Linear Precision Growth  → Store  Redis (GEOADD)
PROTOCOL	WebSockets JetBrains Mono		Use Logic: Bi-Directional / Low Latency  Use  GPS Streams
SERVICES	Request Vehicle, Location Mgr, ETA, Trip Mgr		Logic: Microservices Architecture → 
DATA STORES	Redis (Cache/Loc), Postgres (Trip/Bill), Graph (Nav)		 Postgres (relational database)   ↔  Graph 
ALGORITHMS	A* Pathfinding JetBrains Mono		Logic: Weighted Edges (Traffic) → 